

# Root Finding II: Newton's Method

Tangent lines and quadratic convergence

Sharareh Sayyad

Washington State University

October 7, 2026

# Outline

From brackets to tangents

Newton's method in practice

Quadratic convergence

What can go wrong

Safeguarded Newton

Newton without derivatives

Stopping and cost

Summary

## Recap: bracketing is reliable, but slow

Last lecture: bisection and false position.

- ▶ **Always** converge once we have a bracket.
- ▶ Only **linear**: bisection gains one bit per step,  $\approx 20$  steps for  $10^{-6}$ .
- ▶ Need a **sign change**: they cannot find the root of  $(x - 1)^2$ .

### Today

Use the **derivative**  $f'(x)$  as well as  $f(x)$ . We give up the guarantee and get much more speed:

correct digits roughly **double** every step.

| $\sqrt{2}$ , $\text{tol} = 10^{-6}$ | steps |
|-------------------------------------|-------|
| bisection on $[1, 2]$               | 20    |
| false position on $[1, 2]$          | 8     |
| Newton from $x_0 = 1$               | 4     |

### The trade-off

Newton needs **no bracket**, only a starting guess  $x_0$ . But it can fail if  $x_0$  is poor: it is a **local** method.

# The idea: follow the tangent line

Near  $x_n$ , replace  $f$  by its **tangent line**:

$$\ell(x) = f(x_n) + f'(x_n)(x - x_n).$$

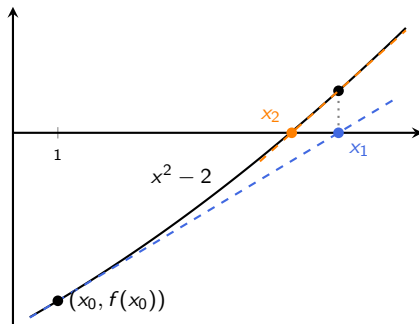
Take the zero of  $\ell$  as the next guess:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

## Compare with false position

False position: zero of a **secant** through two bracket points.

Newton: zero of the **tangent** at one point;  
no bracket is kept.



$x_0 = 1 \rightarrow x_1 = 1.5 \rightarrow x_2 = 1.41\bar{6}$ :  
already within 0.0025 of the root  $\sqrt{2}$ .

## Where the formula comes from: Taylor's theorem

Expand  $f$  around the current guess  $x_n$ :

$$f(x) = f(x_n) + f'(x_n)(x - x_n) + \frac{1}{2}f''(\xi)(x - x_n)^2.$$

- ▶ Drop the quadratic term:  $f(x) \approx \ell(x)$ , the tangent line.
- ▶ Solve  $\ell(x) = 0$  for  $x$ : that is  $x_{n+1}$ .

Newton = solve a linear model, repeat

Each step replaces a hard problem (nonlinear  $f$ ) by an easy one (a line). The neglected term is  $O((x - x_n)^2)$ , which is why the method is so fast once  $x_n$  is close.

### Requirement

We need a formula (or code) for  $f'(x)$ , and  $f'(x_n) \neq 0$ .

# Newton's method in Python

```
def newton(f, df, x0, tol=1e-10, maxit=50):
    x = x0
    for n in range(maxit):
        fx, dfx = f(x), df(x)  # two evaluations
        if dfx == 0:
            raise ZeroDivisionError("f'(x) = 0")
        dx = fx / dfx          # Newton step
        x = x - dx
        if abs(dx) < tol:      # step-size test
            return x
    raise RuntimeError("no convergence")
```

## Raise, don't return

Newton **may not converge**, so hitting `maxit` is an **error**, not a result. Never silently return the last `x`.

## Cost per step

One  $f$  **and** one  $f'$  evaluation. If  $f'$  is expensive or unknown, this matters (see Newton without derivatives).

## Why test the step $|dx|$ ?

No bracket, no error bound, but the Newton step is an excellent error estimate (later).

Example:  $\sqrt{2}$  by Newton,  $f(x) = x^2 - 2$ ,  $x_0 = 1$

| $n$ | $x_n$                    | $ x_n - \sqrt{2} $         | $\frac{ e_n }{ e_{n-1} ^2}$ |
|-----|--------------------------|----------------------------|-----------------------------|
| 0   | 1.0                      | $4.1 \times 10^{-1}$       |                             |
| 1   | 1.5                      | $8.6 \times 10^{-2}$       | 0.500                       |
| 2   | <b>1.416666666666667</b> | $2.5 \times 10^{-3}$       | 0.333                       |
| 3   | <b>1.414215686274510</b> | $2.1 \times 10^{-6}$       | 0.353                       |
| 4   | <b>1.414213562374690</b> | $1.6 \times 10^{-12}$      | 0.354                       |
| 5   | <b>1.414213562373095</b> | $\leq 2.2 \times 10^{-16}$ |                             |

Bold digits are correct.

- ▶ Correct digits: 2, 6, 12, 16. They **double** each step until machine precision.
- ▶ The ratio  $|e_n|/|e_{n-1}|^2$  settles to a constant:

$$C = \frac{1}{2\sqrt{2}} \approx 0.354.$$

- ▶ Compare: bisection needs  $\approx 52$  steps to reach full precision.

# Why the digits double

Let  $e_n = x_n - r$ . Taylor around  $x_n$ , evaluated at the root:

$$0 = f(r) = f(x_n) - f'(x_n) e_n + \frac{1}{2} f''(\xi_n) e_n^2.$$

Divide by  $f'(x_n)$  and use the Newton formula:

$$\underbrace{x_n - \frac{f(x_n)}{f'(x_n)}}_{x_{n+1}} - r = \frac{f''(\xi_n)}{2f'(x_n)} e_n^2$$

$$\implies e_{n+1} \approx \frac{f''(r)}{2f'(r)} e_n^2$$

## Order of convergence $p$

$$|e_{n+1}| \approx C |e_n|^p.$$

- ▶  $p = 1$ : linear (bisection, false position)
- ▶  $p \approx 1.44$ : Illinois
- ▶  $p = 2$ : **quadratic** (Newton)

If  $e_n \approx 10^{-k}$  then  $e_{n+1} \approx C \cdot 10^{-2k}$ : twice as many digits.

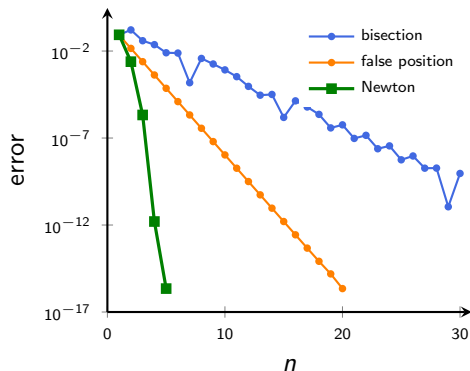
# The convergence theorem: fast, but only locally

## Theorem (local convergence)

Let  $f$  be twice continuously differentiable near  $r$ , with  $f(r) = 0$  and  $f'(r) \neq 0$ . Then there is a  $\delta > 0$  such that for every  $x_0$  with  $|x_0 - r| < \delta$ , Newton's method converges to  $r$  **quadratically**.

Read the fine print:

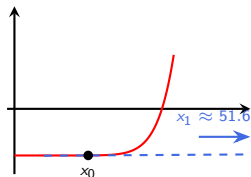
- ▶ “ $x_0$  close enough”: the theorem does **not** say how close.  $\delta$  can be tiny.
- ▶ “ $f'(r) \neq 0$ ”: a **simple** root. Double roots are slower.



$x^2 - 2$ . Linear methods give straight lines on semilogy; Newton's curve **bends down**: quadratic.

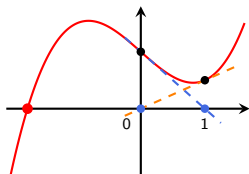
## Newton: what can go wrong

Newton only uses *local* information. Far from the root, the tangent can point anywhere.



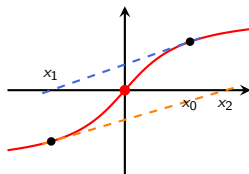
### Flat tangent: overshoot

$f(x) = x^{10} - 1$ ,  $x_0 = 0.5$ :  
 $f'(x_0) \approx 0.02$ , so the step is huge.  
It then takes **42 steps** to crawl back. If  $f'(x_n) = 0$  exactly, the step is undefined.



### Cycling

$f(x) = x^3 - 2x + 2$ ,  $x_0 = 0$ : the iterates go  $0, 1, 0, 1, \dots$  forever. The real root  $r \approx -1.769$  is never reached.



### Divergence

$f(x) = \arctan x$ , root 0. From  $x_0 = 1.5$ :  
 $-1.69, 2.32, -5.11, 32.3, \dots$  Each step overshoots more. From  $x_0 = 1.3$  it converges.

## Multiple roots: Newton slows down to linear

$f(x) = (x - 1)^2$  has a **double root**:  
 $f(1) = f'(1) = 0$ . Newton still converges, but slowly ( $x_0 = 2$ ):

| $n$      | $x_n$ | $ x_n - 1 $ |
|----------|-------|-------------|
| 0        | 2     | 1           |
| 1        | 1.5   | 0.5         |
| 2        | 1.25  | 0.25        |
| 3        | 1.125 | 0.125       |
| $\vdots$ |       | $\vdots$    |

Linear with  $\rho = \frac{1}{2}$ : no better than bisection. Here  $f'(r) = 0$ , so the theorem does not apply.

### Root of multiplicity $m$

If  $f(x) = (x - r)^m h(x)$  with  $h(r) \neq 0$ , then

$$e_{n+1} \approx \left(1 - \frac{1}{m}\right) e_n.$$

Linear, with  $\rho = 1 - \frac{1}{m}$ : worse for higher  $m$ .

### Fix: modified Newton

If  $m$  is known,

$$x_{n+1} = x_n - m \frac{f(x_n)}{f'(x_n)}$$

is quadratic again  $((x - 1)^2, m = 2$ : exact in one step).

Symptom: the step only halves each iteration.

# Best of both: Newton with a bracket

```
def safe_newton(f, df, a, b, tol=1e-10, maxit=100):
    fa = f(a)
    if np.sign(fa) == np.sign(f(b)):
        raise ValueError("not a bracket")
    x = a + (b - a)/2
    for n in range(maxit):
        fx, dfx = f(x), df(x)
        if np.sign(fx) == np.sign(fa):
            a, fa = x, fx          # root in [x, b]
        else:
            b = x                  # root in [a, x]
        if dfx != 0:
            xnew = x - fx/dfx      # try Newton
        if dfx == 0 or not (a <= xnew <= b):
            xnew = a + (b - a)/2   # fall back
        if abs(xnew - x) < tol:
            return xnew
    x = xnew
    return x
```

## Rule

Keep a bracket  $[a, b]$  all the time. Take the Newton step if it lands **inside** the bracket; otherwise take a **bisection** step.

- ▶ **Guaranteed** to converge, like bisection.
- ▶ Near the root, Newton steps land inside: **quadratic**.

| <hr/> $x^{10} - 1$ on $[0, 1.3]$ steps to $10^{-6}$ <hr/> |    |
|-----------------------------------------------------------|----|
| bisection                                                 | 20 |
| false position                                            | 61 |
| Illinois                                                  | 13 |
| Newton, $x_0 = 0.65$                                      | 21 |
| safeguarded Newton                                        | 4  |

Plain Newton from the midpoint jumps to  $x_1 \approx 5.4$ . The safeguard biseects once, then 3 Newton steps.

## Newton without derivatives: approximate the slope

Newton's idea only needs a **slope** at  $x_n$ . If  $f'$  is unknown or expensive, approximate it.

### Option 1: finite-difference Newton

$$f'(x_n) \approx \frac{f(x_n + h) - f(x_n)}{h}$$

with  $h \approx \sqrt{\varepsilon_{\text{mach}}} |x_n| \approx 10^{-8} |x_n|$ . Still **2 evaluations** per step.

### Option 2: reuse the previous point

We already know  $f(x_{n-1})$ . Use it, for **free**:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

This is the **secant method**.

Finite-difference Newton,  $x^2 - 2$ ,  $x_0 = 1$ :

| $n$ | $h = 10^{-8}$         | $h = 10^{-3}$         |
|-----|-----------------------|-----------------------|
| 1   | $8.6 \times 10^{-2}$  | $8.6 \times 10^{-2}$  |
| 2   | $2.5 \times 10^{-3}$  | $2.5 \times 10^{-3}$  |
| 3   | $2.1 \times 10^{-6}$  | $3.0 \times 10^{-6}$  |
| 4   | $1.6 \times 10^{-12}$ | $1.1 \times 10^{-9}$  |
| 5   | $2.2 \times 10^{-16}$ | $3.8 \times 10^{-13}$ |

- ▶  $h = 10^{-8}$ : same errors as exact Newton.
- ▶  $h = 10^{-3}$ : slope error  $\approx h$ , so convergence drops below quadratic.
- ▶  $h$  too small ( $\ll 10^{-8}$ ): rounding in  $f(x_n + h) - f(x_n)$  ruins the slope.

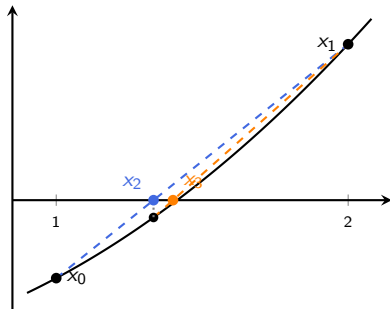
## The secant method

Put the secant slope into Newton's formula:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

- ▶ Needs **two** starting points  $x_0, x_1$ , no bracket, no  $f'$ .
- ▶ Same formula as false position, but always keeps the **two newest** points: no endpoint gets stuck, and no guarantee.
- ▶ Order  $p = \frac{1+\sqrt{5}}{2} \approx 1.618$  with **1 evaluation** per step.

| $\sqrt{2}$ , tol = $10^{-6}$ | steps | $f/f'$ evaluations |
|------------------------------|-------|--------------------|
| Newton, $x_0 = 1$            | 4     | 8                  |
| secant, $x_0 = 1, x_1 = 2$   | 5     | 7                  |



Errors:  $10^{-2} \rightarrow 10^{-4} \rightarrow 10^{-6} \rightarrow 10^{-10} \rightarrow 10^{-16}$ .  
Exponents grow by  $\approx 1.6\times$  per step (Newton:  $2\times$ ).

# The secant method in Python

```
def secant(f, x0, x1, tol=1e-10, maxit=50):
    f0, f1 = f(x0), f(x1)
    for n in range(maxit):
        if f1 == f0:
            raise ZeroDivisionError("f1 = f0")
        dx = f1*(x1 - x0)/(f1 - f0) # secant step
        x0, f0 = x1, f1           # shift
        x1 = x1 - dx
        f1 = f(x1)                # 1 evaluation
        if abs(dx) < tol: # step-size test
            return x1
    raise RuntimeError("no convergence")
```

Given to you in the notebook (Activity, Problem 2), with the list hist of iterates added.

## Same skeleton as newton

Only the step changes:  $f'(x_1)$  is replaced by the slope  $\frac{f_1 - f_0}{x_1 - x_0}$ . Same step-size test, same raise at maxit.

## Order matters

Compute dx with the **old** points, then shift, then update x1.  $f(x_0)$  is kept in f0, never recomputed: **1 evaluation** per step.

## A small step is not a root

$x^{10} - 1$  from  $x_0 = 0$ ,  $x_1 = 1.3$ : secant stops at  $x \approx 0.18$ , where  $|f(x)| \approx 1$ . Check the residual  $|f(x)|$  too.

## When do we stop?

No bracket, so no guaranteed error bound. But quadratic convergence makes the **step size** a very good error estimate:

$$x_{n+1} - x_n = e_{n+1} - e_n \approx -e_n \quad \text{since } |e_{n+1}| \ll |e_n|.$$

So  $|x_{n+1} - x_n| \approx |e_n|$ , and  $x_{n+1}$  is **even more accurate** than that.

### Practical recipe

- ▶ Plot  $f$ ; pick  $x_0$  near the root (or a bracket).
- ▶ Stop when  $|x_{n+1} - x_n| < \text{tol} \cdot |x_{n+1}|$ .
- ▶ Cap with `maxit`; **raise** if reached.

### Floating point floor

Once  $|e_n| \approx 10^{-16}|r|$ , iterates only jiggle by rounding. Don't ask for more.

## Comparison: bracketing vs. Newton vs. secant

|                        | bisection | false pos. | Newton    | secant       |
|------------------------|-----------|------------|-----------|--------------|
| needs                  | bracket   | bracket    | $x_0, f'$ | $x_0, x_1$   |
| always converges       | yes       | yes        | <b>no</b> | <b>no</b>    |
| order $p$              | 1         | 1          | <b>2</b>  | 1.618        |
| evaluations per step   | 1         | 1          | 2         | 1            |
| order per evaluation   | 1         | 1          | 1.41      | <b>1.618</b> |
| error bound in advance | yes       | no         | no        | no           |
| double roots           | no        | no         | linear    | linear       |

### Counting work fairly

Newton does 2 evaluations per step, so per **evaluation** its order is  $2^{1/2} \approx 1.41$ . The secant method (1.618) is the most efficient per evaluation. Newton wins when  $f'$  is cheap: it often shares most of its work with  $f$ .

### Rule of thumb

Reliability from brackets, speed from Newton: in practice, combine them.

# Summary

## Newton's method

- ▶  $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$
- ▶ zero of the tangent line
- ▶ needs  $x_0$  and  $f'$ , no bracket

## Convergence

- ▶  $e_{n+1} \approx \frac{f''(r)}{2f'(r)} e_n^2$
- ▶ quadratic: digits double
- ▶ only if  $x_0$  is close and  $f'(r) \neq 0$
- ▶ multiple root: linear

## No derivative

- ▶ finite differences,  $h \approx 10^{-8}|x|$
- ▶ secant: slope from the last two points
- ▶ order 1.618, 1 evaluation per step

## In practice

- ▶ can overshoot, cycle, diverge
- ▶ safeguard with a bracket
- ▶ stop on step size, cap maxit
- ▶ raise an error if it fails

Newton's idea: fastest when it works, so start it close or protect it with a bracket.

Next session: root finding with `scipy.optimize (newton, brentq, root_scalar)`.